



FICHE DE CONCEPTION FONCTIONNELLE – EXTRADATA

1. OBJECTIF DU PROJET

Le projet **Extradata** a pour objectif de simplifier et d'automatiser l'extraction de données essentielles depuis des fichiers PDF.

De nombreux professionnels et étudiants perdent un temps considérable à parcourir manuellement leurs documents pour en extraire des informations clés (montants, dates, noms, adresses, etc.).

L'application permet de glisser-déposer un ensemble de fichiers PDF, de sélectionner les champs souhaités, puis de générer automatiquement un fichier CSV contenant les données structurées et exploitables.

Extradata se positionne comme un outil rapide, intuitif et accessible, destiné à améliorer la productivité et à éviter les erreurs humaines.

2. PUBLIC VISE / UTILISATEUR TYPE

L'application s'adresse à tout utilisateur ayant besoin de manipuler ou d'exploiter des informations contenues dans des fichiers PDF, sans passer par une saisie manuelle :

- Étudiants manipulant des relevés, rapports ou documents de recherche.
- Professionnels (comptables, secrétaires, gestionnaires, etc.) traitant des factures ou devis.
- Développeurs et data analysts cherchant à extraire automatiquement des données pour un usage statistique.

Extradata vise donc un **public large**, à la fois technique et non technique.

3. FONCTIONNALITES PRINCIPALES

- **Glisser-déposer de fichiers PDF** : l'utilisateur peut importer un ou plusieurs fichiers à la fois.
- **Interface simple et interactive** : sélection des champs à extraire (mots-clés, valeurs numériques, dates, etc.).
- **Extraction automatique** : traitement des fichiers PDF via des bibliothèques Python spécialisées.



- **Exportation en CSV** : génération automatique d'un tableau de résultats prêt à être analysé.
- **Journal des extractions** : affichage des fichiers traités et des éventuelles erreurs détectées.
- **Sauvegarde automatique** : conservation des configurations et des historiques d'extraction pour une réutilisation future.

4. TECHNOLOGIES UTILISEES

Le projet **Extradata** s'appuie sur un ensemble de technologies open-source fiables et modernes :

- **Python** : langage principal pour la logique de traitement et la création de l'interface.
- **pdfplumber / PyPDF2** : pour l'extraction du texte et des métadonnées des fichiers PDF.
- **Tkinter ou customtkinter** : pour la conception d'une interface graphique ergonomique et intuitive.
- **Pandas** : pour la gestion, le formatage et l'export des données en CSV.
- **PyInstaller** : pour transformer le projet Python en une application exécutable (.exe) prête à l'emploi.

5. ARCHITECTURE GLOBALE DU CODE

L'application **Extradata** sera structurée de manière modulaire afin d'assurer la clarté, la maintenabilité et la possibilité d'évolution future (ajout de nouveaux types de documents, de formats d'export, etc.).

L'objectif est de séparer clairement la **logique métier**, la **gestion des fichiers**, et l'**interface utilisateur**.

```
extradata/
|
|—— main.py                # Point d'entrée principal de l'application
|
|—— gui/                   # Gestion de l'interface utilisateur
|   |—— __init__.py
|   |—— main_window.py     # Interface principale (drag & drop, boutons, menus)
|   |—— extraction_window.py # Écran de sélection des champs
|   |—— progress_window.py  # Écran de chargement et suivi d'extraction
```



```
|   |—— assets/                # Images, icônes, logo
|
|—— core/                    # Logique métier et traitements
|   |—— __init__.py
|   |—— extractor.py          # Module d' extraction PDF
|   |—— parser.py             # Traitement du texte, regex, nettoyage
|   |—— csv_generator.py      # Génération et export des fichiers CSV
|   |—— config.py             # Gestion des paramètres et constantes globales
|
|—— utils/                   # Fonctions utilitaires
|   |—— __init__.py
|   |—— file_manager.py       # Gestion des chemins, renommage, vérification
|   |—— logger.py             # Gestion des logs et erreurs
|
|—— tests/                   # Dossier de tests unitaires
|   |—— test_extractor.py
|   |—— test_parser.py
|   |—— test_csv_generator.py
|
|—— requirements.txt          # Liste des dépendances Python
|—— README.md                 # Documentation d' installation et d' utilisation
|—— setup.py                  # Script d' installation (si packaging)
```

Principes clés de conception :

- **Séparation des responsabilités** : chaque module a un rôle bien défini.
- **Extensibilité** : ajout possible de nouvelles catégories de documents sans modifier le cœur du programme.
- **Clarté du flux de données** :
 - GUI → sélection utilisateur
 - core.extractor → extraction du texte brut
 - core.parser → identification des champs
 - core.csv_generator → export des données
- **Utilisation d'exceptions personnalisées** pour gérer les erreurs (fichiers corrompus, champs non trouvés, etc.).
- **Interface non bloquante** : utilisation de threads pour que la fenêtre reste réactive durant les extractions.

6. INTERFACE UI/UX.





Extradala

Extraction des données Lancer l'extraction

- ☒ Montant Total ☐
- ☐ Date de Facture ☐
- ☒ Numéro de Facture ☐
- ☐ Nom du Client ☐
- ☐ Adresse ☐
- ☒ Postes Détaillés ☐

Fichier source: facture_012023.pdf

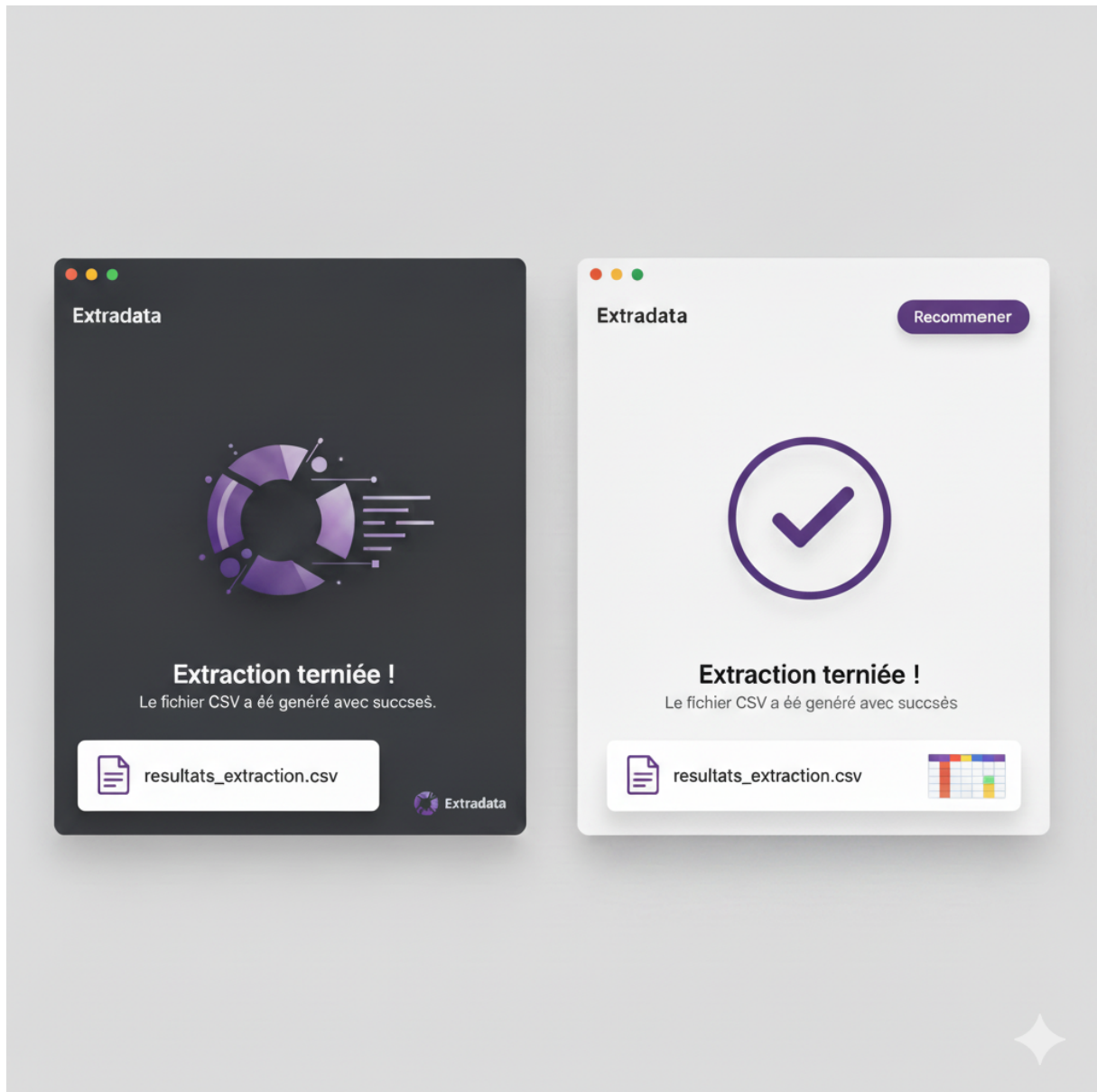
Extradala

Extraction des données Lancer l'extraction

- ☒ Montant Total ☐
- ☐ Date de Facture ☐
- ☒ Numéro de Facture ☐
- ☐ Nom du Client ☐
- ☐ Adresse ☐
- ☒ Postes Détaillés ☐

Fichier source: facture_012023.pdf





7. LIVRABLES ATTENDUS

À la fin du développement du projet **Extradata**, les éléments suivants seront fournis :

1. Application fonctionnelle :

- Version exécutable (pour MacOS dans un premier temps, et après Windows.).
- Interface graphique complète (écrans d'accueil, sélection, extraction).
- Fonctionnalités principales opérationnelles :
 - Import de plusieurs fichiers PDF.
 - Sélection des champs à extraire.



- Extraction automatique et génération de CSV.

2. Code source complet et documenté :

- Arborescence de code conforme au plan d'architecture.
- Commentaires explicatifs pour chaque module.
- Fichier requirements.txt listant les dépendances.
- README.md clair (installation, utilisation, exemples).

3. Données de test et exemples :

- Ensemble de fichiers PDF d'exemple (factures, CVs, etc.).
- Exemple de CSV généré par l'application.
- Logs de test démontrant le fonctionnement correct des modules.

4. Documentation technique :

- Schéma d'architecture du code.
- Description du flux de traitement (du drag & drop à l'export CSV).
- Liste des fonctions principales et des classes.

5. Livrables visuels :

- Maquettes UI (mode clair/sombre).
- Logo officiel **Extradata** au format PNG/SVG.
- Captures d'écran de la version finale.